Java 객체지향 핵심 정리

1. 객체의 설계와 생성
2. 데이터 보호와 은닉
3. 재사용과 확장

Made with GAMMA

CHAPTER 1

객체의 설계와 생성

1.1 변수의 종류

지역 변수, 클래스 변수, 객체 변수의 차이와 메모리 구조

1.2 생성자

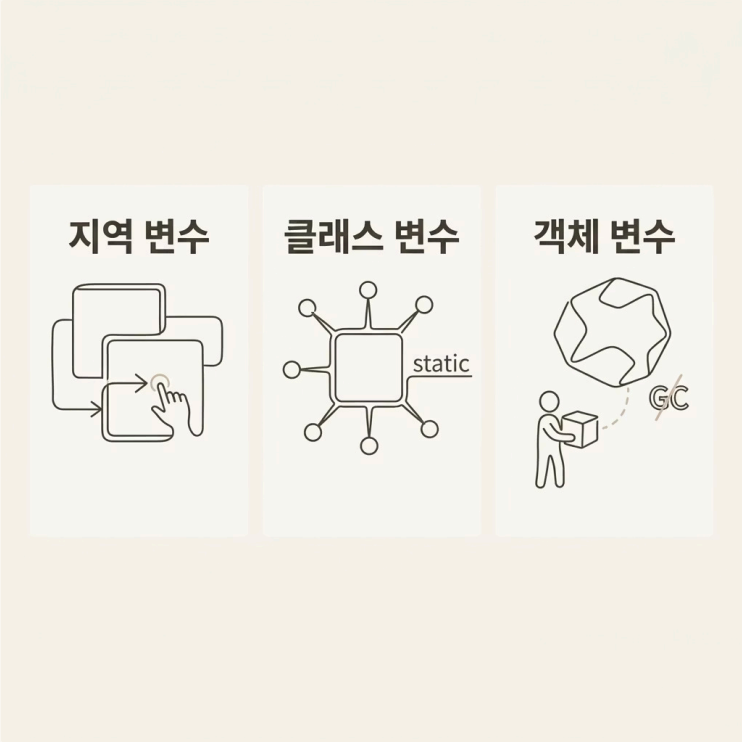
객체 초기화를 담당하는 단 한 번 호출되는 특별한 메서드

1.3 객체 배열

여러 객체를 묶어 관리하는 배열 — 선언과 초기화의 차이 주의

Made with GAMMA

변수의 종류



핵심 비교

지역 변수

메서드 호출 시 생성 → 반환 시 소멸. 스택 메모리 사용

클래스 변수

static 키워드 선언. 모든 메서드 공유, 프로그램 내내 유지

객체 변수

new로만 생성, 힙 메모리 사용. 참조 소멸 시 GC 대상

```
public class Practice {
  static int classVar = 1; // 클래스 변수
  String instanceVar; // 객체 변수

  public static void main(String[] args) {
    int localVar = 0; // 지역 변수
    Practice a = new Practice();
  }
}
```

Made with GAMMA

생성자 (Constructor)

생성자는 `new`에 의해 단 한 번 호출되며, 객체 변수의 초기값을 설정하는 특별한 메서드입니다.

→ 클래스 이름과 동일

생성자의 이름은 반드시 클래스 이름과 같아야 한다

→ 반환 타입 없음

`void` 포함 어떤 반환 타입도 명시하지 않는다

→ 기본 생성자 자동 제공

직접 정의하지 않으면 컴파일러가 매개변수 없는 기본 생성자를 자동으로 추가

```
public class Keroro {
  private String 이름;

  public Keroro(String 이름) { // 생성자: 클래스명과 동일, 반환타입 없음
    this.이름 = 이름;
  }

  public static void main(String[] args) {
    Keroro a = new Keroro("피카츄"); // 단 한 번 호출
  }
}
```

Made with GAMMA

객체 배열

⚠️ 가장 흔한 실수

배열 선언과 객체 초기화는 **별개의 작업**입니다. 자리만 만들고 값을 넣지 않으면 `NullPointerException`이 발생합니다.

Step 1 — 자리 만들기

new Practice[5] → 객체 주소를 담을 공간 5개 생성

Step 2 — 객체 넣기

new Practice("피카츄") → 실제 객체를 생성하여 배열에 저장

객체 배열은 **객체의 주소(참조)**를 저장하는 배열이다!

```
public class Practice {
  String 이름;

  public Practice(String 이름) {
    this.이름 = 이름;
  }

  public static void main(String[] args) {
   // Step 1: 자리만 만든 상태 (null로 가득 찬 배열)
     Practice[] 객체배열 = new Practice[5];

   // Step 2: 실제 객체 생성 후 배열에 저장
     객체배열[0] = new Practice("피카츄");
     객체배열[1] = new Practice("파이리");
  }
}
```

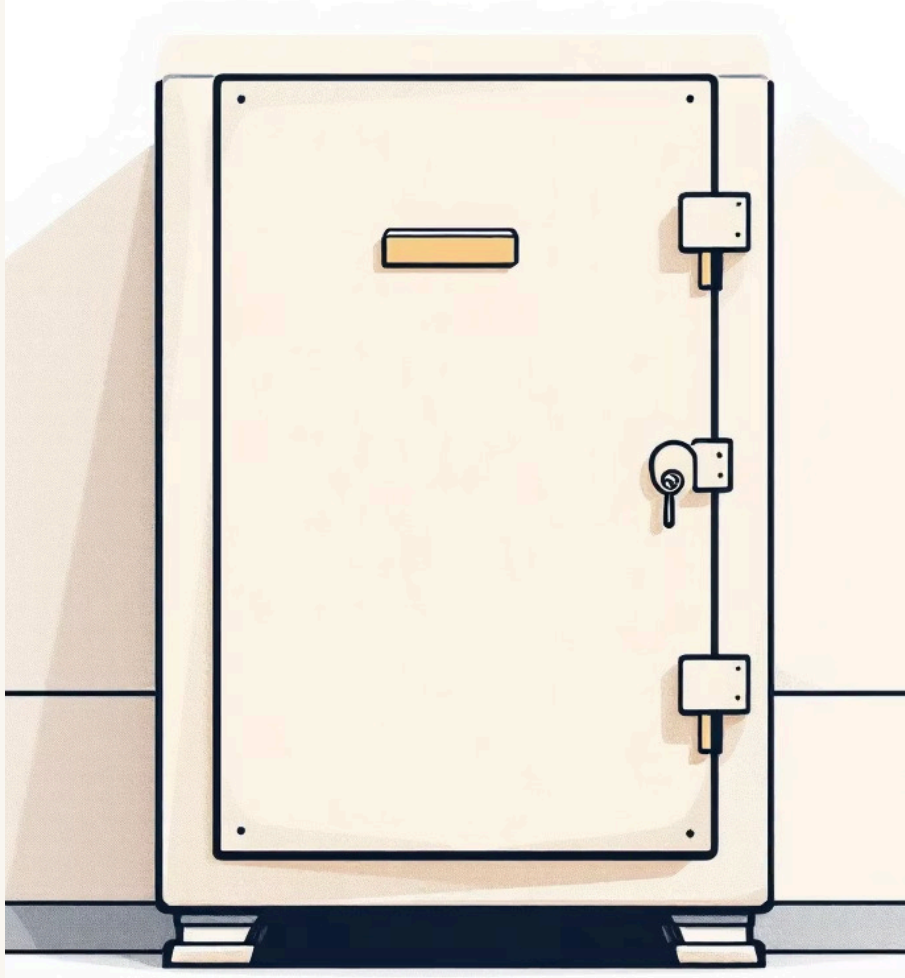
Made with GAMMA

CHAPTER 2

데이터 보호와 은닉

캡슐화 · 접근 제한자

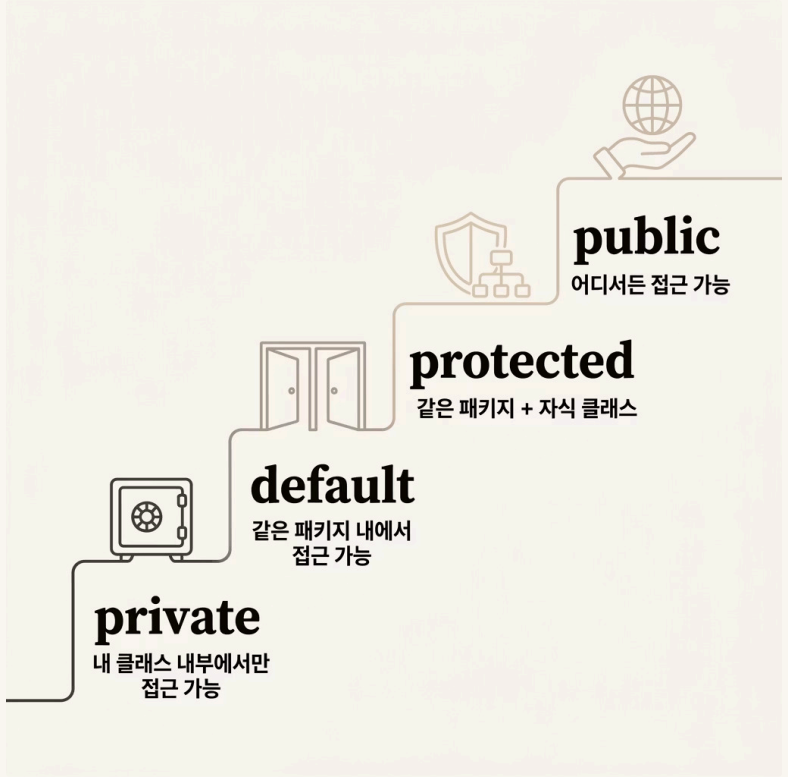
데이터를 외부로부터 보호하고, 잘못된 값이 들어가는 것을 원천 차단하는 객체지향의 핵심 원칙



Made with GAMMA

캡슐화와 접근 제한자

접근 제한자 범위



캡슐화 예제 코드

```
public class Member {
    // 1. 데이터 은닉: 외부에서 직접 접근하지 못하도록
    //private 설정
    private String name;
    private int age;

    // 2. 접근 허용: public 메서드(Getter/Setter)를 통해
    //데이터에 접근
    public String getName() {
        return name;
    }

    public void setAge(int age) {
        if (age >= 0) {
        // 데이터 검증 로직 추가 가능 (캡슐화의 장점)
            this.age = age;
        }
    }
}
```

Made with GAMMA

CHAPTER 3

재사용과 확장 — 상속

자식 클래스가 부모 클래스의 필드와 메서드를 **물려받아** 재사용하고, 필요한 기능을 **추가·재정의**하여 확장하는 것

단일 상속

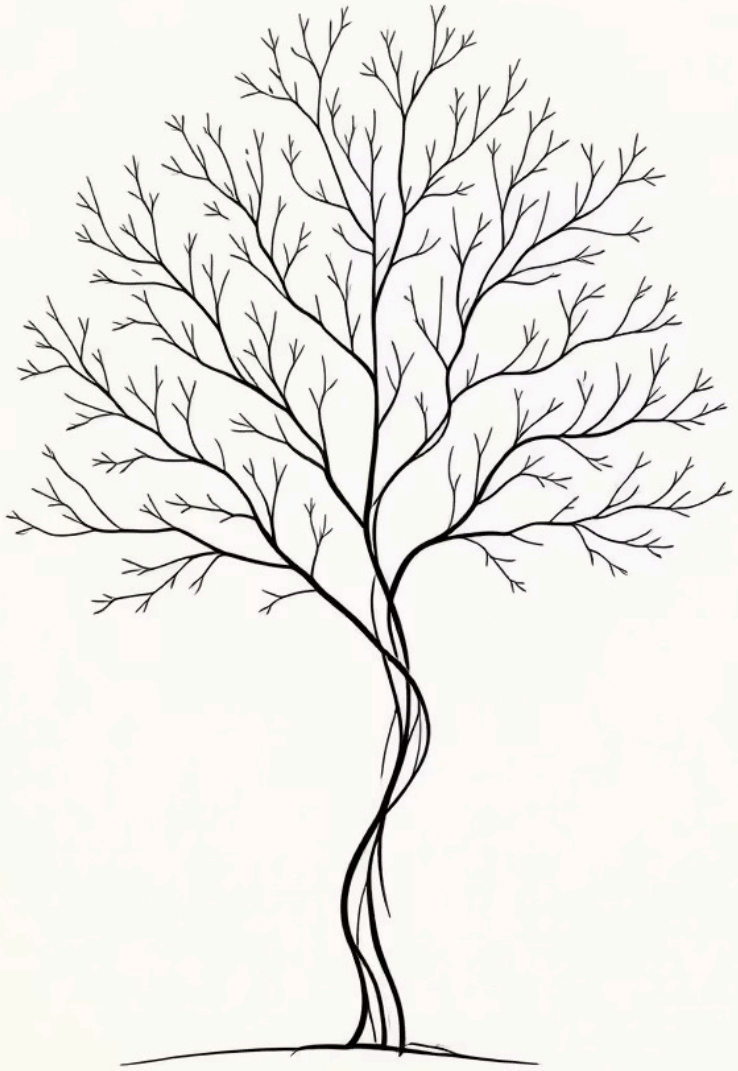
Java는 하나의 부모 클래스만 상속 가능

상속 키워드: extends

자식 클래스 선언에 extends 부모클래스 작성

부모 호출: super()

자식 생성자에서 super()를 사용하여 부모 생성자 호출



Made with GAMMA

❌ 상속 없이 — 중복 발생

```
// Professor 클래스
class Professor {
    String name; int age; int stress;
    public Professor(String name, int age, int stress) { ... }
    public void exercise() { stress -= 20; }
    public void eat()     { stress -= 20; }
    public void personInfo() { ... }
    public void teach()   { stress += 10; }
}

// Student 클래스 — 거의 동일한 코드 반복!
class Student {
    String name; int age; int stress;
    public Student(String name, int age, int stress) { ... }
    public void exercise() { stress -= 20; }
    public void eat()     { stress -= 20; }
    public void personInfo() { ... }
    public void study()   { stress += 40; }
}
```

✅ 상속 사용 — 중복 제거

```
// 공통 필드·메서드는 부모 클래스에만
class Person {
    String name; int age; int stress;
    public Person(String name, int age, int stress) {
        this.name = name; this.age = age;
        this.stress = stress;
    }
    public void exercise() { stress -= 20; }
    public void personInfo() {
        System.out.println("name : " + name);
    }
}

// Student: 공통 부분 상속 + 고유 기능만 추가
class Student extends Person {
    public Student(String name, int age, int stress) {
        super(name, age, stress); // 부모 생성자 호출
    }
    public void study() { stress += 40; }
}

// Professor: 메서드 오버라이딩 가능
class Professor extends Person {
    public Professor(String name, int age, int stress) {
        super(name, age, stress);
    }
    @Override
    public void exercise() { // 재정의
        System.out.println("운동합니다."); stress -= 100;
    }
    public void teach() { stress += 10; }
}
```

Made with GAMMA

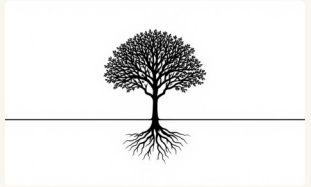
핵심 정리

객체 설계와 생성

지역·클래스·객체 변수의 메모리 위치를 이해하고, 생성자로 초기화, 객체 배열은 **선언 후 반드시 초기화**

캡슐화와 접근 제한자

변수는 `private`으로 숨기고, getter/setter로만 접근. **캡슐화**는 개념, **접근 제한자**는 구현 도구



상속 (extends)

공통 코드는 부모에 한 번만 작성. `super()`로 부모 생성자 호출, `@Override`로 기능 재정의

Made with GAMMA